

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12417158
Kind Code	B2
Date of Patent	September 16, 2025
Inventor(s)	Sheperek; Michael et al.

Closing block family based on soft and hard closure criteria

Abstract

A system includes a memory device and a processing device, operatively coupled to the memory device. The processing device initializes a block family associated with the memory device and a timer associated with the block family. In response to the timer reaching a soft closure value, the processing device performs a soft closure of the block family and a hard closure of the block family in response to a first of the timer reaching a hard closure value or the block family satisfying a hard closure criteria. The processing device detects a first write amplification of a previously closed block family that employed the hard closure value and increases the hard closure value by an amount of time calculated to reduce a second write amplification, of the block family, to below a threshold write amplification.

Inventors: Sheperek; Michael (Longmont, CO), Koudele; Larry J. (Erie, CO), Williams; Steven S. (Longmont, CO)

Applicant: Micron Technology, Inc. (Boise, ID)

Family ID: 1000008818314

Assignee: Micron Technology, Inc. (Boise, ID)

Appl. No.: 18/431101

Filed: February 02, 2024

Prior Publication Data

Document Identifier	Publication Date
US 20240168670 A1	May. 23, 2024

Related U.S. Application Data

continuation parent-doc US 17868008 20220719 US 11940892 child-doc US 18431101
division parent-doc US 16947712 20200813 US 11429504 20220830 child-doc US 17868008

Publication Classification

Int. Cl.: G06F11/30 (20060101); G06F3/06 (20060101); G06F11/07 (20060101); G06F11/10 (20060101); G06F11/16 (20060101)

U.S. Cl.:

CPC G06F11/3037 (20130101); G06F3/0619 (20130101); G06F3/064 (20130101); G06F3/0679 (20130101); G06F11/0757 (20130101); G06F11/076 (20130101); G06F11/1004 (20130101); G06F11/1658 (20130101);

Field of Classification Search

CPC: G06F (3/0619); G06F (3/064); G06F (3/0679); G06F (11/3037); G06F (11/0757); G06F (11/076); G06F (11/1004); G06F (11/1658)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
9640281	12/2016	Seo et al.	N/A	N/A
9858002	12/2017	Goss	N/A	G06F 3/0619
11210031	12/2020	Mekhanik et al.	N/A	N/A
11573720	12/2022	Sheperek et al.	N/A	N/A
2003/0046020	12/2002	Scheuerlein	N/A	N/A
2017/0255403	12/2016	Sharon et al.	N/A	N/A
2018/0033491	12/2017	Marelli	N/A	G11C 16/26
2018/0293029	12/2017	Achtenberg et al.	N/A	N/A
2020/0051621	12/2019	Papandreou et al.	N/A	N/A
2022/0050758	12/2021	Sheperek	N/A	G06F 11/3037
2022/0057934	12/2021	Sheperek	N/A	G06F 3/064
2022/0229554	12/2021	Muchherla	N/A	G11C 11/5628
2022/0300166	12/2021	Kientz et al.	N/A	N/A
2022/0350718	12/2021	Sheperek	N/A	G06F 11/1658
2023/0030620	12/2022	Lee	N/A	G06F 3/0652
2023/0280926	12/2022	Kelner	N/A	G06F 3/0619
2023/0386578	12/2022	Xu	N/A	G11C 16/08

OTHER PUBLICATIONS

Kim J., et al., "Adaptive Delay Monitoring for Wide Voltage-Range Operation," Design, Automation & Test in Europe Conference & Exhibition, 2016, pp. 511-516.
cited by applicant

Primary Examiner: Bertram; Ryan

Attorney, Agent or Firm: Lowenstein Sandler LLP

Background/Summary

RELATED APPLICATIONS (1) The present application is a continuation of U.S. patent application Ser. No. 17/868,008, filed Jul. 19, 2022, which is a divisional of U.S. patent application Ser. No. 16/947,712, filed Aug. 13, 2020 (now U.S. Pat. No. 11,429,504), both of which are incorporated by reference herein in their entireties.

TECHNICAL FIELD

(1) Embodiments of the disclosure are generally related to memory sub-systems, and more specifically, are related to closing a block family based on soft and hard closure criteria.

BACKGROUND

(2) A memory sub-system can include one or more memory devices that store data. The memory devices can be, for example, non-volatile memory devices and volatile memory devices. In general, a host system can utilize a memory sub-system to store data at the memory devices and to retrieve data from the memory devices.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- (1) The present disclosure will be understood more fully from the detailed description given below and from the accompanying drawings of some embodiments of the disclosure.
- (2) FIG. 1 illustrates an example computing system that includes a memory sub-system in accordance with some embodiments.
- (3) FIG. 2 is a set of graphs that illustrate a temporal voltage shift caused by the slow charge loss exhibited by triple-level memory cells, in accordance with some embodiments.
- (4) FIG. 3 is an example graph illustrating the dependency of a threshold voltage offset on the time after program, e.g., the period of time elapsed since the a page had been programmed, in accordance with some embodiments.
- (5) FIG. 4 is an example graph illustrating the dependency of the threshold voltage offset on both time after program and average temperature, in accordance with some embodiments.
- (6) FIG. 5 is a graph that illustrates a set of predefined threshold voltage offset bins, in accordance with embodiments.
- (7) FIG. 6 is a block diagram that illustrates operation of a block family manager within the memory sub-system controller of FIG. 1 in accordance with various embodiments.
- (8) FIG. 7 is a flow diagram of an example method for closing, by a memory sub-system controller, an open block family using time-based soft closure criteria, in accordance with some embodiments.
- (9) FIG. 8 is a flow diagram of an example method of closing, by a memory sub-system controller, an open block family using temperature-based soft closure criteria, in accordance with some embodiments.
- (10) FIG. 9 is a flow diagram of an example method for determining, by a memory sub-system controller, hard closure criteria for use in performing a hard closure of a block family according to some embodiments.
- (11) FIG. 10 is a block diagram of an example computer system in which embodiments of the present disclosure can operate.

DETAILED DESCRIPTION

(12) Embodiments of the present disclosure are directed to closing a block family based on soft and hard closure criteria. A memory sub-system can be a storage device, a memory module, or a hybrid of a storage device and memory module. Examples of storage devices and memory modules are described below in conjunction with FIG. 1. In general, a host system can utilize a memory sub-

system that includes one or more components, such as memory devices that store data. The host system can provide data to be stored at the memory sub-system and can request data to be retrieved from the memory sub-system.

(13) A memory sub-system can utilize one or more memory devices, including any combination of the different types of non-volatile memory devices and/or volatile memory devices, to store the data provided by the host system. In some embodiments, non-volatile memory devices can be provided by negative-and (NAND) type flash memory devices. Other examples of non-volatile memory devices are described below in conjunction with FIG. 1. A non-volatile memory device is a package of one or more dice. Each die can consist of one or more planes. Planes can be groups into logic units (LUN). For some types of non-volatile memory devices (e.g., NAND devices), each plane consists of a set of physical blocks. A “block” herein refers to a set of contiguous or non-contiguous memory pages. An example of a block is an erasable block, which is a minimal erasable unit of memory, while a page is a minimal writable unit of memory. Each page includes a set of memory cells (“cells”). A cell is an electronic circuit that stores information.

(14) Data operations can be performed by the memory sub-system. The data operations can be host-initiated operations. For example, the host system can initiate a data operation (e.g., write, read, erase, etc.) on a memory sub-system. The host system can send access requests (e.g., write command, read command) to the memory sub-system, such as to store data on a memory device at the memory sub-system and to read data from the memory device on the memory sub-system. The data to be read or written, as specified by a host request, is hereinafter referred to as “host data.” A host request can include logical address information (e.g., logical block address (LBA), namespace) for the host data, which is the location the host system associates with the host data. The logical address information (e.g., LBA, namespace) can be part of metadata for the host data. Metadata can also include error handling data (e.g., ECC codeword, parity code), data version (e.g. used to distinguish age of data written), valid bitmap (which LBAs or logical transfer units contain valid data), block family information, and the like.

(15) A memory device includes multiple memory cells, each of which can store, depending on the memory cell type, one or more bits of information. A memory cell can be programmed (written to) by applying a certain voltage to the memory cell, which results in an electric charge being held by the memory cell, thus allowing modulation of the voltage distributions produced by the memory cell. Moreover, precisely controlling the amount of the electric charge stored by the memory cell allows to establish multiple threshold voltage levels corresponding to different logical levels, thus effectively allowing a single memory cell to store multiple bits of information: a memory cell operated with $2^{\text{sup.}n}$ different threshold voltage levels is capable of storing n bits of information. “Threshold voltage” herein shall refer to the voltage level that defines a boundary between two neighboring voltage distributions corresponding to two logical levels. Thus, the read operation can be performed by comparing the measured voltage exhibited by the memory cell to one or more reference voltage levels in order to distinguish between two logical levels for single-level cells and between multiple logical levels for multi-level cells.

(16) Due to the phenomenon known as slow charge loss, the threshold voltage of a memory cell changes in time as the electric charge of the cell degrades, which is referred to as “temporal voltage shift” (TVS), since the degrading electric charge causes the voltage distributions to shift along the voltage axis towards lower voltage levels. The threshold voltage is changing rapidly at first (immediately after the memory cell was programmed), and then slows down in an approximately logarithmic linear fashion with respect to the time elapsed since the cell programming event. Slow charge loss can also increase with increasing temperature of memory cells as well as with increasing program erase cycles, among other factors. For example, if over a week, a memory device remains close to 0° C., the slow charge loss can be around 20-50 millivolts (mV), while if the memory device is around 100° C., the slow charge loss can be around 400 mV, a significant difference. Accordingly, failure to mitigate the temporal voltage shift caused by the slow charge

loss can result in the increased bit error rate in read operations.

(17) However, various common implementations either fail to adequately address the temporal voltage shift or employ inefficient strategies resulting in high bit error rates and/or exhibiting other shortcomings. Embodiments of the present disclosure address the above-noted and other deficiencies by implementing a memory sub-system that employs block family based error avoidance strategies, thus significantly improving the bit error rate exhibited by the memory sub-system.

(18) According to various embodiments, the temporal voltage shift is selectively tracked for a programmed set of memory cells grouped by block families, and appropriate voltage offsets, which are based on block affiliation with a certain block family, are applied to the base read levels in order to perform read operations. “Block family” herein shall refer to a possibly non-contiguous set of memory cells (which can reside in one or more full and/or partial blocks, the latter referred to as “partitions” herein) that have been programmed within a specified time window and a specified temperature window, and thus are expected to exhibit similar or correlated changes in their respective data state metrics. A block family can be made with any granularity, containing only whole codewords, whole pages, whole super pages, or whole superblocks, or any combination of these. “Data state metric” herein shall refer to a quantity that is measured or inferred from the state of data stored on a memory device. Specifically, the data state metrics can reflect the state of the temporal voltage shift, the degree of read disturb, and/or other measurable functions of the data state. A composite data state metric is a function (e.g., a weighted sum) of a set of component state metrics.

(19) In these embodiments, a block family experiences a specified temperature range using an aggregate temperature, which can change or be defined according to a temperature metric in different embodiments. For example, in response to a timer reaching a soft closure value, a controller of the memory sub-system can perform a soft closure of the block family. Or, alternatively, in response to an aggregate temperature (based on temperature values received from temperature sensor(s) of the memory device) being greater than or equal to a specified threshold temperature, the controller can likewise perform a soft closure of the block family.

(20) After performing the soft closure, the memory sub-system can continue to program to a partially written block until the timer reaches hard closure value or the block family satisfies a hard closure criteria. In this way, the memory sub-system has an extension time before the hard closure during which to complete writing to the partially written block, decide whether to partition the block, or to write dummy data to the unwritten portion of the block after hard closure. After hard closure of one block family, further programming to dice of the memory device is to a newly opened block family. Since the time elapsed after programming and temperature are the main factors affecting the temporal voltage shift, all blocks and/or partitions within a single block family are presumed to exhibit similar distributions of threshold voltages in memory cells, and thus would require the same voltage offsets to be applied to the base read levels for read operations. “Base read level” herein shall refer to the initial threshold voltage level exhibited by the memory cell immediately after programming. In some implementations, base read levels can be stored in the metadata of the memory device.

(21) Block families can be created asynchronously with respect to block programming events. In an illustrative example, a new block family can be created whenever the timer reaches the hard closure value or a value of the aggregate temperature is greater than or equal to a specified threshold temperature value. Because slow charge loss is impacted by the passage of time and temperature variation, time and/or temperature can be used for determining the duration of an open block family before the block family is closed, as will be discussed in detail.

(22) The controller can periodically perform a calibration process in order to associate each die of every block family with one of multiple predefined threshold voltage offset bins, which is in turn associated with the voltage offset to be applied for read operations. The associations of pages or

blocks with block families and block families and dies with threshold voltage offset bins can be stored in respective metadata tables maintained by the memory sub-system controller.

(23) Upon receiving a read command, the memory sub-system controller can identify the block family associated with the page or block identified by the logical block address (LBA) specified by the read command, identify the threshold voltage offset bin associated with the block family and die on which the page or block resides, compute the new threshold voltage by additively applying the threshold voltage offset associated with the threshold voltage offset bin to the base read level, and perform the read operation using the new threshold voltage. In some embodiments, the threshold voltage offset bin can be selected at least in part based on a reference temperature of blocks in the block family. The reference temperature can represent a temperature value of the memory cells of the block family as a whole.

(24) Therefore, advantages of the systems and methods implemented in accordance with some embodiments of the present disclosure include, but are not limited to, improving the bit error rate in read operations by maintaining metadata tracking groups of memory cells (as block families) that are presumed to exhibit similar voltage distributions and selectively performing calibration operations for limited subsets of memory cells based on their block family association. Further, because such tracking, storage, and calibration are performed on a block family basis as opposed to a per-block (or per-page) basis, processing, memory, and storage resources are preserved for host system usage. Other advantages will be apparent to those skilled in the art of memory allocation and error optimization within a memory sub-system discussed hereinafter.

(25) FIG. 1 illustrates an example computing system **100** that includes a memory sub-system **110** in accordance with some embodiments of the present disclosure. The memory sub-system **110** can include media, such as one or more volatile memory devices (e.g., memory device **140**), one or more non-volatile memory devices (e.g., memory device **130**), or a combination of such.

(26) A memory sub-system **110** can be a storage device, a memory module, or a hybrid of a storage device and memory module. Examples of a storage device include a solid-state drive (SSD), a flash drive, a universal serial bus (USB) flash drive, an embedded Multi-Media Controller (eMMC) drive, a Universal Flash Storage (UFS) drive, a secure digital (SD) card, and a hard disk drive (HDD). Examples of memory modules include a dual in-line memory module (DIMM), a small outline DIMM (SO-DIMM), and various types of non-volatile dual in-line memory module (NVDIMM).

(27) The computing system **100** can be a computing device such as a desktop computer, laptop computer, network server, mobile device, a vehicle (e.g., airplane, drone, train, automobile, or other conveyance), Internet of Things (IOT) enabled device, embedded computer (e.g., one included in a vehicle, industrial equipment, or a networked commercial device), or such computing device that includes memory and a processing device (e.g., a processor).

(28) The computing system **100** can include a host system **120** that is coupled to one or more memory sub-systems **110**. In some embodiments, the host system **120** is coupled to different types of memory sub-systems **110**. FIG. 1 illustrates one example of a host system **120** coupled to one memory sub-system **110**. As used herein, “coupled to” or “coupled with” generally refers to a connection between components, which can be an indirect communicative connection or direct communicative connection (e.g., without intervening components), whether wired or wireless, including connections such as electrical, optical, magnetic, etc.

(29) The host system **120** can include a processor chipset and a software stack executed by the processor chipset. The processor chipset can include one or more cores, one or more caches, a memory controller (e.g., NVDIMM controller), and a storage protocol controller (e.g., PCIe controller, SATA controller). The host system **120** uses the memory sub-system **110**, for example, to write data to the memory sub-system **110** and read data from the memory sub-system **110**.

(30) The host system **120** can be coupled to the memory sub-system **110** via a physical host interface. Examples of a physical host interface include, but are not limited to, a serial advanced

technology attachment (SATA) interface, a peripheral component interconnect express (PCIe) interface, universal serial bus (USB) interface, Fibre Channel, Serial Attached SCSI (SAS), a double data rate (DDR) memory bus, Small Computer System Interface (SCSI), a dual in-line memory module (DIMM) interface (e.g., DIMM socket interface that supports Double Data Rate (DDR)), Open NAND Flash Interface (ONFI), Double Data Rate (DDR), Low Power Double Data Rate (LPDDR), etc. The physical host interface can be used to transmit data between the host system **120** and the memory sub-system **110**. The host system **120** can further utilize an NVM Express (NVMe) interface to access components (e.g., memory devices **130**) when the memory sub-system **110** is coupled with the host system **120** by the PCIe interface. The physical host interface can provide an interface for passing control, address, data, and other signals between the memory sub-system **110** and the host system **120**. FIG. 1 illustrates a memory sub-system **110** as an example. In general, the host system **120** can access multiple memory sub-systems via a same communication connection, multiple separate communication connections, and/or a combination of communication connections.

(31) The memory devices **130,140** can include any combination of the different types of non-volatile memory devices and/or volatile memory devices. The volatile memory devices (e.g., memory device **140**) can be, but are not limited to, random access memory (RAM), such as dynamic random access memory (DRAM) and synchronous dynamic random access memory (SDRAM).

(32) Some examples of non-volatile memory devices (e.g., memory device **130**) include negative- and (NAND) type flash memory and write-in-place memory, such as a three-dimensional cross-point (“3D cross-point”) memory device, which is a cross-point array of non-volatile memory cells. A cross-point array of non-volatile memory can perform bit storage based on a change of bulk resistance, in conjunction with a stackable cross-gridded data access array. Additionally, in contrast to many flash-based memories, cross-point non-volatile memory can perform a write in-place operation, where a non-volatile memory cell can be programmed without the non-volatile memory cell being previously erased. NAND type flash memory includes, for example, two-dimensional NAND (2D NAND) and three-dimensional NAND (3D NAND).

(33) Each of the memory devices **130** can include one or more arrays of memory cells. One type of memory cell, for example, single level cells (SLC) can store one bit per cell. Other types of memory cells, such as multi-level cells (MLCs), triple level cells (TLCs), and quad-level cells (QLCs), can store multiple bits per cell. In some embodiments, each of the memory devices **130** can include one or more arrays of memory cells such as SLCs, MLCs, TLCs, QLCs, or any combination of such. In some embodiments, a particular memory device can include an SLC portion, and an MLC portion, a TLC portion, or a QLC portion of memory cells. The memory cells of the memory devices **130** can be grouped as pages that can refer to a logical unit of the memory device used to store data. With some types of memory (e.g., NAND), pages can be grouped to form blocks.

(34) Although non-volatile memory devices such as 3D cross-point array of non-volatile memory cells and NAND type memory (e.g., 2D NAND, 3D NAND) are described, the memory device **130** can be based on any other type of non-volatile memory, such as read-only memory (ROM), phase change memory (PCM), self-selecting memory, other chalcogenide based memories, ferroelectric transistor random-access memory (FeTRAM), ferroelectric random access memory (FeRAM), magneto random access memory (MRAM), Spin Transfer Torque (STT)-MRAM, conductive bridging RAM (CBRAM), resistive random access memory (RRAM), oxide based RRAM (OxRAM), negative-or (NOR) flash memory, and electrically erasable programmable read-only memory (EEPROM).

(35) A memory sub-system controller **115** (or controller **115** for simplicity) can communicate with the memory devices **130** to perform operations such as reading data, writing data, or erasing data at the memory devices **130** and other such operations. The memory sub-system controller **115** can

include hardware such as one or more integrated circuits and/or discrete components, a buffer memory, or a combination thereof. The hardware can include digital circuitry with dedicated (i.e., hard-coded) logic to perform the operations described herein. The memory sub-system controller **115** can be a microcontroller, special purpose logic circuitry (e.g., a field programmable gate array (FPGA), an application specific integrated circuit (ASIC), etc.), or other suitable processor.

(36) The memory sub-system controller **115** can include a processor **117** (e.g., processing device) configured to execute instructions stored in a local memory **119**. In the illustrated example, the local memory **119** of the memory sub-system controller **115** includes an embedded memory configured to store instructions for performing various processes, operations, logic flows, and routines that control operation of the memory sub-system **110**, including handling communications between the memory sub-system **110** and the host system **120**.

(37) In some embodiments, the local memory **119** can include memory registers storing memory pointers, fetched data, etc. The local memory **119** can also include read-only memory (ROM) for storing micro-code. While the example memory sub-system **110** in FIG. **1** has been illustrated as including the controller **115**, in another embodiment of the present disclosure, a memory sub-system **110** does not include a controller **115**, and can instead rely upon external control (e.g., provided by an external host, or by a processor or controller separate from the memory sub-system).

(38) In general, the memory sub-system controller **115** can receive commands or operations from the host system **120** and can convert the commands or operations into instructions or appropriate commands to achieve the desired access to the memory devices **130**. The memory sub-system controller **115** can be responsible for other operations such as wear leveling operations, garbage collection operations, error detection and error-correcting code (ECC) operations, encryption operations, caching operations, and address translations between a logical address (e.g., logical block address (LBA), namespace) and a physical address (e.g., physical block address) that are associated with the memory devices **130**. The memory sub-system controller **115** can further include host interface circuitry to communicate with the host system **120** via the physical host interface. The host interface circuitry can convert the commands received from the host system into command instructions to access the memory devices **130** as well as convert responses associated with the memory devices **130** into information for the host system **120**.

(39) In some implementations, memory sub-system **110** can use a striping scheme, according to which every the data payload (e.g., user data) utilizes multiple dies of the memory devices **130** (e.g., NAND type flash memory devices), such that the payload is distributed through a subset of dies, while the remaining one or more dies are used to store the error correction information (e.g., parity bits). Accordingly, a set of blocks distributed across a set of dies of a memory device using a striping scheme is referred herein to as a “superblock.”

(40) The memory sub-system **110** can also include additional circuitry or components that are not illustrated. In some embodiments, the memory sub-system **110** can include a cache or buffer (e.g., DRAM) and address circuitry (e.g., a row decoder and a column decoder) that can receive an address from the controller **115** and decode the address to access the memory devices **130**.

(41) In some embodiments, the memory devices **130** include local media controllers **135** that operate in conjunction with memory sub-system controller **115** to execute operations on one or more memory cells of the memory devices **130**. An external controller (e.g., memory sub-system controller **115**) can externally manage the memory device **130** (e.g., perform media management operations on the memory device **130**). In some embodiments, a memory device **130** is a managed memory device, which is a raw memory device combined with a local controller (e.g., local controller **135**) for media management within the same memory device package. An example of a managed memory device is a managed NAND (MNAND) device.

(42) The memory sub-system **110** includes a block family manager **113** that can be used to implement the block family-based error avoidance strategies in accordance with embodiments of

the present disclosure. In some embodiments, the controller **115** includes at least a portion of the block family manager **113**. For example, the controller **115** can include a processor **117** (processing device) configured to execute instructions stored in local memory **119** for performing the operations described herein. In some embodiments, the block family manager **113** is part of the host system **120**, an application, or an operating system. The block family manager **113** can manage block families associated with the memory devices **130**, as described in more detail herein below.

(43) FIG. 2 is a set of graphs that illustrate a temporal voltage shift caused by the slow charge loss exhibited by triple-level memory cells, in accordance with some embodiments. While the illustrative example of FIG. 2 utilizes triple-level cells, the same observations can be made and, accordingly, the same remedial measures are applicable to single level cells and multi-level cells in order to compensate for the slow charge loss.

(44) As noted herein above, a memory cell can be programmed (written to) by applying a certain voltage to the memory cell, which results in an electric charge being held by the memory cell, thus allowing modulation of the voltage distributions produced by the memory cell. Precisely controlling the amount of the electric charge stored by the memory cell allows to establish multiple threshold voltage levels corresponding to different logical levels, thus effectively allowing a single memory cell to store multiple bits of information: a memory cell operated with $2^{\text{sup.}n}$ different threshold voltage levels is capable of storing n bits of information.

(45) In FIG. 2, each graph illustrates a voltage distribution produced by memory cells programmed by a respective write level (which can be assumed to be at the midpoint of the distribution) to encode a corresponding logical level (“000” through “111” in case of a TLC). In order to distinguish between neighboring distributions (corresponding to two different logical levels), the threshold voltage levels (shown by dashed vertical lines) are defined, such that any measured voltage that falls below a threshold level is associated with one distribution of the pair of neighboring distributions, while any measured voltage that is greater than or equal to the threshold level is associated with another distribution of the pair of neighboring distributions.

(46) The set of a graphs include a first graph **210** that reflects a time period immediately after programming and a second graph **230** that reflects a long time after programming. As seen by comparing the second graph **230** to the first graph **210**, the voltage distributions change in time due to the slow charge loss, which results in drifting values of the threshold voltage levels (shown by dashed vertical lines). In various embodiments, this temporal voltage shift (TVS) is selectively tracked for programmed pages or blocks grouped by block families, and appropriate voltage offsets, which are based on page or block affiliation with a certain block family, are applied to the base read levels in order to perform read operations.

(47) FIG. 3 is an example graph **300** illustrating the dependency of a threshold voltage offset **310** on the time after program **320**, e.g., the period of time elapsed since a page had been programmed, in accordance with some embodiments. FIG. 4 is an example graph **400** illustrating the dependency of the threshold voltage offset on both time after program (TAP) and average temperature, in accordance with some embodiments. As schematically illustrated by FIG. 3, page or blocks (or groups of memory cells at another granularity) of the memory device are grouped into block families **330A-330N**, such that each block family includes one or more pages or blocks that have been programmed within a specified time window, potentially varied by average temperature while the block family is open (FIG. 4). As noted herein above, since the time elapsed after programming and temperature are the main factors affecting the temporal voltage shift, all pages, blocks, and/or partitions within a single block family are presumed to exhibit similar distributions of threshold voltages in memory cells, and thus would require the same voltage offsets for read operations as time passes.

(48) Block families can be created asynchronously with respect to page programming events. In an illustrative example, the memory sub-system controller **115** of FIG. 1 can create a new block family whenever a specified period of time (e.g., a predetermined number of minutes) has elapsed

since creation of the last block family, which time period can vary significantly depending on an average temperature associated with pages during programming. More specifically, the entire asymptotic curve illustrated in FIG. 3 can be shifted to have a steeper curve with respect to time, as illustrated in FIG. 4, as average temperature increases. In FIG. 4, the curve associated with 20° C. decreases with time at a much slower rate (e.g., about 100 times slower) compared to the curve associated with 88° C. For example, the former exits Bin 7 in excess of 100,000 hours TAP while the latter exits Bin 7 at about 1,000 hours TAP. The curves in FIG. 4 look differently from the curve in FIG. 3 due to being graphed at log 10 scale in order to illustrate the difference in slow charge loss as temperature varies. Slow charge loss is illustrated along the vertical access for the seventh valley (V7) based on digital-to-analog (DAC) converted voltage values, also referred to as DACs. Each DAC can represent a certain number of millivolts (mV), here about 10 mV.

(49) A newly created block family can be associated with bin 0, and each subsequently created block family can be associated with a sequentially numbered block family. Then, the memory sub-system controller can periodically perform a calibration process in order to associate each die of every block family with one of the predefined threshold voltage offset bins (e.g., bins 0-7 in the illustrative example of FIG. 3), which is in turn associated with the voltage offset to be applied for read operations. The associations of pages, blocks, and/or partitions with block families and block families and dies with threshold voltage offset bins can be stored in respective metadata tables maintained by the memory sub-system controller 115.

(50) FIG. 5 is a graph that illustrates a set of predefined threshold voltage offset bins (bin 0 to bin 9), in accordance with some embodiments. As schematically illustrated by FIG. 5, the threshold voltage offset graph can be subdivided into multiple threshold voltage offset bins, such that each bin corresponds to a predetermined range of threshold voltage offsets. While the illustrative example of FIG. 5 defines ten bins, in other implementations, various other numbers of bins can be employed (e.g., 16, 32, 64 bins). Based on a periodically performed calibration process, the memory sub-system controller 115 can associate each die of every block family with a threshold voltage offset bin, which defines a set of threshold voltage offsets to be applied to the base voltage read level in order to perform read operations.

(51) FIG. 6 is a block diagram that illustrates operation of the block family manager 113 within the memory sub-system controller 115 of FIG. 1 in accordance with various embodiments. In various embodiment, the memory sub-system can include temperature sensors 602 and a thermocouple 604. The thermocouple 604 can be coupled over at least a portion of the memory device 130 to help cool the memory device 130. The memory device 130 can include multiple dice 132, including a first die 132A, a second die 132B, . . . to include an Nth die 132N. The temperature sensors 602 can include, but not be limited to, a temperature sensor coupled to the controller 115, a temperature sensor coupled to the thermocouple 604, a temperature sensor located elsewhere within the memory sub-system 110, a temperature sensor at one die, of multiple dice of the memory device 130, and/or temperature sensors distributed across each of the multiple dice 132 of the memory device 130.

(52) In this way, the block family manager 113 can determine a temperature for or associated with a block family within the memory device 130 in different ways. For example, the block family manager 113 can use a temperature measured of the thermocouple 604, of the controller 115, of another location within the memory sub-system 110, to include the temperature of one die or of the multiple dice 132 of the memory device 130. In some embodiments, the inter-temperature difference between the multiple dice 132 is small, e.g., within a few degrees; in other embodiments, the temperature difference is large, e.g., within tens of degrees. Further, if a temperature is measured at one or more of the multiple dice 132, the reference temperature for the memory device 130 can be determined at the die with the maximum temperature across the multiple dice 132.

(53) The block family manager 113 can include a timer 612, a temperature tracker 614, and an extension timer 618, although other functionality of the block family manager 113 will be

discussed with reference to managing and tracking block families throughout this disclosure. In alternative embodiments, the timer **612** and/or the extension timer **618** is located elsewhere in the controller **115**, such as within the processor **117**, to which the block family manager **113** has access. The local memory **119** can store a number of different items of information or data that will be discussed in more detail, including but not limited to, a block family (BF) start time **622**, temperature metrics **626**, an active block family (BF) identifier **628**, BF soft closure criteria **630**, BF hard closure criteria **636**, reference temperatures **634**, cursors **638** associated with the memory device **130**, a temporal shift function **640**, and a set of metadata tables **650**. This information and data can be flushed to the memory device **130** (or other non-volatile memory) in response to detection of an imminent loss of power.

(54) More specifically, in various embodiments, the block family manager **113** can open a new block family after a previously block family has been closed. At initiation of each block family, the block family manager **113** can initialize the timer **612** associated with a system clock. The system clock, for example, can be a clock maintained by the memory sub-system **110** and/or the host system **120**. The time at which the block family is opened on the system clock can be stored as the BF start time **622**. The block family manager **113** can further, using one or more of the temperature sensor(s) **602**, measure an opening temperature of the memory device **130**. This opening temperature can be stored in the local memory **119**, such as with values of the temperature metrics **626**.

(55) In one embodiment, the block family manager **113**, in response to the timer reaching a soft closure value, performs a soft closure of the block family. The soft closure value on the timer, for example, can be a predetermined amount of time such as 50 minutes, 60 minutes, 70 minutes or the like determined to be a sufficient amount of time to capture a number of pages or blocks without a high write amplification while also keeping the bit error rate of reads to the block family low. Write amplification is increased with tracking too much metadata due to the block families being too small. The soft closure value can be stored as part of the BF soft closure criteria **630**.

(56) As time passes while the controller **115** programs the BF of the memory device **130**, the block family manager **113** (e.g., the temperature tracker **614**) can continuously calculate values for one or more of the temperature metrics **626** based on temperature values received from one or more of the temperature sensor(s) **602**, to include an aggregate temperature across multiple dice of the memory device **130**. For example, the block family manager can aggregate the temperature values received from one or more temperature sensors of the memory device over time to determine an aggregate temperature. In this way, the values for the temperature metrics **626** are tracked and a history of the temperature metrics can also be stored in the reference temperatures **634** of the local memory **119**. The block family manager **113** can then compare the temperature metric values against specified threshold temperature values that function as the BF soft closure criteria **630** for closing the block family. For example, in an alternative embodiment, in response to determining that the aggregate temperature is greater than or equal to a specified threshold temperature value, the block family manager **113** can perform a soft closure of the block family. These various metrics and values will be discussed in more detail.

(57) The block family currently being programmed can be referred to as the active block family for which the active BF identifier **628** can be stored in the local memory **119** and indexed to within the metadata tables **650**. For case of tracking, each subsequently programmed block family can be assigned a sequentially numbered BF identifier, but other sequences and identifiers can also be used. The blocks families can also be associated with one or more of the cursors **638**, e.g., at least an opening cursor and a closing cursor for each BF. "Cursor" herein shall broadly refer to a location on the memory device to which the data is being written.

(58) In some embodiments, the soft closure of the block family signals to the block family manager **113** that the open time of the block family is nearing an end, and a decision is to be made about how and when to perform a hard closure of the block family. During this time, the controller **115**

can continue to program data to a partially written block that has already been associated with the block family. The block family manager **113** can perform a hard closure of the block family in response to the timer **612** reaching a hard closure value or the block family satisfying a hard closure criteria. Evaluating the hard closure criteria to determine how to hard close the block family will be discussed in more detail with reference to FIG. 9.

(59) In various embodiments, the block family manager **113** sets the hard closure value of the timer based on at least one of a workload of the processing device (e.g., the controller **115**), a reference temperature associated with the block family, or a performance expectation of the processing device. In the alternative, the block family manager **113** sets the hard closure value of the timer **612** as the soft closure value plus an additional threshold percentage of the soft closure value. For example, the additional threshold percentage can be 5%, 10%, 15% or some other percentage of the soft closure value. The hard closure value for the timer and the hard closure criteria can be stored as the BF hard closure criteria **636** in the local memory **119**.

(60) In some embodiments, the block family manager **113** can also adjust the hard closure value depending on detected bit error rates, write amplification, or other error handling statistics experienced with reference to previous block families. For example, the block family manager **113** can detect a bit error rate of a previously closed block family that employed the hard closure value, where the bit error rate is above a threshold acceptable bit error rate. The block family manager **113** can further reduce the hard closure value by an amount of time calculated to reduce the bit error rate to below the threshold acceptable bit error rate. For example, the temporal voltage shift (TVS) function **640** is a function that describes how the TVS value measurement (or other data state metric) varies as a function of time after program. By correlating the TVS function **640** with bit error rate for the block family over time, the block family manager **113** can calculate a reduction in time after program that reduces the bit error rate by a certain amount for the block family.

(61) In another embodiment, the block family manager **113** detects a write amplification of a previously closed block family that employed the hard closure value, where the write amplification is above a threshold acceptable write amplification. The block family manager can further increase the hard closure value by an amount of time calculated to reduce the write amplification to below the threshold acceptable write amplification. Similarly, by correlating the TVS function **640** with write amplification over time, the block family manager **113** can calculate an increase in time after program that reduces the write amplification by a certain amount for the block family.

(62) If the BF soft closure criteria **630** is based on temperature, the block family manager **113**, in response to performing a soft closure of the block family, can also initialize the extension timer **618**. The extension timer **618** can provide a way to track additional time left before the block family manager **113** is forced to perform a hard closure of the block family. More specifically, in response to the extension timer **612** reaching an extension time value or the block family satisfying a hard closure criteria, the block family manager can perform a hard closure of the block family, after which data to be programmed is programmed to a new block family. The hard closure criteria will be discussed in more detail with reference to FIG. 9. In embodiments, the block family manager **113** sets the extension time value based on at least one of a workload of the processing device, an immediate reference temperature for the block family, or a performance expectation of the processing device. The extension time value can be stored in the BF hard closure criteria **636** in the local memory **119**.

(63) In some embodiments, the block family manager **113** can also adjust the extension time value depending on detected bit error rates or write amplification experienced with reference to previous block families. For example, the block family manager **113** can detect a bit error rate of a previously closed block family that employed the extension time value, where the bit error rate is above a threshold acceptable bit error rate. The block family manager **113** can further reduce the extension time value by an amount of time calculated to reduce the bit error rate to below the threshold acceptable bit error rate. In an alternative embodiment, the block family manager **113**

detects a write amplification of a previously closed block family that employed the extension time value, where the write amplification is above a threshold acceptable write amplification. The block family manager can further increase the extension time value by an amount of time calculated to reduce the write amplification to below the threshold acceptable write amplification.

(64) In various embodiments, the temperature metrics **626** can include an absolute temperature difference between the maximum (or highest) temperature and a minimum (or lowest) temperature associated with the block family before the block family is closed. The temperature metrics **626** can further include a value derived from integrating, over time, the absolute temperature difference between the opening temperature and an immediate temperature of the memory device **130**. This temperature integration can yield an average absolute temperature difference up to the point the immediate temperature is measured.

(65) The temperature metrics **626** can further include values that represent the average of any of other temperature metrics across multiple measurements, e.g., different temperature sensors **602**, including those positioned at the multiple dice **132** of the memory device **130**. In one embodiment, the block family manager **113** can determine an aggregated temperature metric value over the multiple dice **132** of the memory device. In response to determining that a first temperature metric value for the first die **132A** (or other die), of the multiple dice **132**, is greater than or equal to a maximum threshold temperature value, the block family manager **113** can exclude the first temperature metric value from the aggregated temperature metric value. The aggregated temperature metric value can then be compared against the specified threshold temperature value used as criteria for closing the block family. In this way, an outlier higher temperature value at a die that is above some predefined maximum temperature is excluded and the remaining total temperature metric value is more representative of the overall temperature of multiple dice **132** as a whole.

(66) In various embodiments, the block family manager **113** can set or adjust the BF soft closure criteria **630**, which can vary depending on application or design. The BF soft closure criteria **630** can include, but not be limited to, a single threshold temperature value (e.g., 30° C., 40° C., or the like). The BF soft closure criteria **630** can further be a threshold temperature function that varies with a reference temperature for the block family. For an example of the latter, if the block family was programmed at a high temperature, the threshold temperature value set for closing the block family can also be set higher. Thus, the block family manager can determine the specified threshold temperature value by evaluating a function of an absolute temperature value for a particular reference temperature associate with the block family.

(67) A reference temperature as referred to herein is a temperature that is representative of the block family, and can be determined at different times while the block family is opened, when the block family is closed, as well as after the block family has been closed. If a temperature value is measured for each page when the block is programmed, the reference temperature can be an average of the measured temperature values for the pages or blocks of the block family, which can be referred to as a reference program temperature. The reference temperature can also be recorded as the opening temperature for the block family when the block family is opened. This opening reference temperature value can also be measured at a variety of the temperature sensors **602** as was discussed previously. A further reference temperature can be continuously updated throughout the time the block family is open, e.g., based on measured temperature values while additional pages or blocks are being programmed. Thus, an updated reference temperature value can be calculated during programming the block family. A closing reference temperature can be designated for this further reference temperature value as the block family is closed or close to being closed.

(68) In various embodiments, a temporal voltage shift (TVS) value (or other data state metric) can be measured in one more memory cell of the block family. A data state metric can be a quantity that is measured or inferred from the behavior of data within the memory cells of the block family that

provides insight into the voltage state of the data. Thus, the data state metric can indicate the state of TVS, a 5.sup.th valley location, a 7.sup.th distribution median voltage, a degree of read disturb, the number of error bits measured using a particular read level, or the like value. The block family manager **113** can compare the measured TVS value with the TVS function **640** stored in the local memory **119** to determine a time after program (TAP) for the block family. The TVS function is a function that describes how the TVS value measurement (or other data state metric) varies as a function of time after program. Thus, reference to the TVS value along with change in temperature can, with reference to the TVS function, further inform how much time the block family manager **113** waits until closing the block family.

(69) Furthermore, a threshold voltage offset calculator of the block family manager **113** can access the metadata tables **650** to determine which threshold voltage offset bin should be used for a particular read. As discussed with reference to FIGS. 3-4, as the TVS value for a BF shifts with increasing slow charge loss, the BF will be read with reference to bins with larger voltage offset values. These larger voltage offset values can come from higher-numbered bins, for example.

(70) In one embodiment, after the block family is closed, the block family manager **113** can record a temperature value based on one or more reference temperature for the block family. The block family manager can also read data from any page or block of the block family via application of a threshold voltage offset, from a threshold voltage offset bin associated with the temperature value, to a base read level voltage. In this way, the temperature value directs which threshold voltage offset bin is used because of the significant impact that temperature has on slow large loss.

(71) FIG. 7 is a flow diagram of an example method **700** for closing, by a memory sub-system controller, an open block family using time-based soft closure criteria, in accordance with some embodiments. The method **700** can be performed by processing logic that can include hardware (e.g., processing device, circuitry, dedicated logic, programmable logic, microcode, hardware of a device, integrated circuit, etc.), software (e.g., instructions run or executed on a processing device), or a combination thereof. In some embodiments, the method **700** is performed by the block family manager **113** of FIG. 1. Although shown in a particular sequence or order, unless otherwise specified, the order of the operations can be modified. Thus, the illustrated embodiments should be understood only as examples, and the illustrated operations can be performed in a different order, while some operations can be performed in parallel. Additionally, one or more operations can be omitted in some embodiments. Thus, not all illustrated operations are required in every embodiment, and other process flows are possible.

(72) At operation **710**, the processing logic initializes a block family associated with a memory device, such as the memory device **130** in FIG. 6. The processing logic can record a BF identifier and corresponding one or more cursor that defines the beginning of the block family within the memory device.

(73) At operation **715**, the processing logic initializes a timer associated with the block family, e.g., upon initialization of the block family. This timer can be the timer **612** discussed in referenced to FIG. 6, and can be associated with a system clock.

(74) At operating **720**, responsive to beginning to program a block residing in the memory device, the processing logic associates the block with the block family. This association can also be a partition of a block if less than the entire block is written before the block family is closed, as will be discussed with reference to FIG. 9. Similarly, multiple blocks can be associated with the block family before it is closed.

(75) At operation **725**, the processing logic determines whether the timer has reached a soft closure value. The soft closure value, as discussed, can be a predetermined amount of time such as 50 minutes, 60, minutes, 70 minutes, or other time period. The processing logic continues to wait until the timer reaches the soft closure value or, alternatively, for an aggregate temperature of the memory device to be greater than or equal to a threshold temperature value.

(76) In response to the timer reaching the soft closure value, the processing logic, at operation **730**,

performs a soft closure of the block family. The soft closure provides the processing logic an extension time during which to decide when and how to perform a hard closure on the block family.

(77) At operation **735**, the processing logic continues to program data to the block. The data, for example, can be received from a host system. Programming can continue in order to, if possible, completely fill the block with data before performing a hard closure.

(78) At operation **740**, the processing logic performs a hard closure of the block family in response to one of the timer reaching a hard closure value or the block family reaching a hard closure criteria. The hard closure value can be determined and updated as per the discussion with reference to FIG. **6**. The hard closure criteria may vary depending on application, but examples of the hard closure criteria will be discussed with reference to FIG. **9**.

(79) FIG. **8** is a flow diagram of an example method **800** of closing, by a memory sub-system controller, an open block family using temperature-based soft closure criteria, in accordance with some embodiments. The method **800** can be performed by processing logic that can include hardware (e.g., processing device, circuitry, dedicated logic, programmable logic, microcode, hardware of a device, integrated circuit, etc.), software (e.g., instructions run or executed on a processing device), or a combination thereof. In some embodiments, the method **800** is performed by the block family manager **113** of FIG. **1**. Although shown in a particular sequence or order, unless otherwise specified, the order of the operations can be modified. Thus, the illustrated embodiments should be understood only as examples, and the illustrated operations can be performed in a different order, while some operations can be performed in parallel. Additionally, one or more operations can be omitted in some embodiments. Thus, not all illustrated operations are required in every embodiment, and other process flows are possible.

(80) At operation **810**, the processing logic initializes a block family associated with a memory device, such as the memory device **130** in FIG. **6**. The memory device can include multiple dice that are programmable. The processing logic can record a BF identifier and corresponding one or more cursor that defines the beginning of the block family within the memory device.

(81) At operation **815**, the processing logic can aggregate temperature values received from one or more temperature sensors of the memory device over time. The temperature being aggregated can be derived at some location of the memory sub-system **110**, such as at the controller **115**, a thermocouple, or at one or more of multiple dice of the memory device. In one embodiment, to determine the aggregate temperature, the processing logic measures an opening temperature of the memory device at initialization of the block family. The processing logic further determines a temperature metric value by integrating, over time, an absolute temperature difference between the opening temperature and an immediate temperature of the memory device.

(82) At operation **820**, responsive to beginning to program a block residing in the memory device, the processing logic associates the block with the block family. This association can also be a partition of a block if less than the entire block is written before the block family is closed, as will be discussed with reference to FIG. **9**. Similarly, multiple blocks can be associated with the block family before it is closed.

(83) At operation **825**, the processing logic determines whether the aggregate temperature is greater than or equal to a specified threshold temperature value. The aggregate temperature may be any of the temperature metric values discussed with reference to FIG. **6**. The processing logic can wait for the aggregate temperature to reach the threshold temperature value, or in the alternative, for the timer to reach the soft closure time.

(84) At operation **830**, in response to the aggregate temperature being greater than or equal to the specified threshold temperature value, the processing logic performs a soft closure of the block family. The soft closure provides the processing logic an extension time during which to decide when and how to perform a hard closure on the block family.

(85) At operation **835**, the processing logic initializes an extension timer, e.g., the extension timer

618 of block family manager **113**. The extension timer can track the additional time granted the processing logic before being forced to perform a hard closure of the block family.

(86) At operation **840**, the processing logic continues to program data to the block. The data, for example, can be received from a host system. Programming can continue in order to, if possible, completely fill the block with data before performing a hard closure.

(87) At operation **845**, the processing logic performs a hard closure of the block family in response to one of the extension timer reaching an extension time value or the block family reaching a hard closure criteria. The extension time value can be determined and updated as per the discussion with reference to FIG. **6**. The hard closure criteria may vary depending on application, but examples of the hard closure criteria will be discussed with reference to FIG. **9**.

(88) FIG. **9** is a flow diagram of an example method **900** for determining, by a memory sub-system controller, hard closure criteria for use in performing a hard closure of a block family according to some embodiments. The method **900** can be performed by processing logic that can include hardware (e.g., processing device, circuitry, dedicated logic, programmable logic, microcode, hardware of a device, integrated circuit, etc.), software (e.g., instructions run or executed on a processing device), or a combination thereof. In some embodiments, the method **900** is performed by the block family manager **113** of FIG. **1**. Although shown in a particular sequence or order, unless otherwise specified, the order of the operations can be modified. Thus, the illustrated embodiments should be understood only as examples, and the illustrated operations can be performed in a different order, while some operations can be performed in parallel. Additionally, one or more operations can be omitted in some embodiments. Thus, not all illustrated operations are required in every embodiment, and other process flows are possible.

(89) At operation **905**, the processing logic determines an amount of time until filling the block completely with data being received from a host system. This determination can be made based on a known rate of programming and a known amount of data that is queued to be written from the host system, for example.

(90) At operation **910**, the processing logic decides whether to finish programming the block based on the amount of time (determined at operation **905**) and a remainder of time left until the timer reaches the hard closure value. In the alternative embodiment (FIG. **8**), the remainder of the time left is until the extension timer reaches the extension time value.

(91) If the answer is yes at operation **910**, the processing logic, at operation **915**, finishes programming the block with the data received from the host system before performing the hard closure of the block family. Operation **915** thus can complete creation of a block family and the next action could be to create a new block family to which further data is to be programmed.

(92) If the answer is no at operation **910**, the processing logic, at operation **920**, determines whether to leave empty the unwritten portion of the block. The decision whether to leave the unwritten portion empty can depend on application and/or configuration. For example, if the processing logic is capable of creating and tracking partitions, and thus some sub-block granularity of the memory device, then the unwritten portion can be left empty. Otherwise, the processing logic can write dummy data to the unwritten portion.

(93) More specifically, if the answer is no at operation **920**, the processing logic, at operation **925**, performs a hard closure of the block family. Although this means the unwritten portion of the block can no longer be programmed with host data, the processing logic can, at operation **930**, write dummy data to the unwritten portion of the block. In this way, the block is forced to be completely written and no partial block (or sub-block) tracking is necessary for the block family. The dummy data can be, for example, all zero values (0s) or all one values (1s).

(94) If the answer is yes at operation **920**, the processing logic, at operation **935**, creates a first partition of a written portion of the block. At operation **940**, the processing logic associates the first partition with the block family. At operation **945**, the processing logic creates a second partition of the unwritten portion of the block for association with a new block family. At operation **945**, the

processing logic, at operation **925**, performs a hard closure of the block family.

(95) In some embodiments, if the processing logic writes a small amount of data to the memory device over time, the processing logic can create additional partitions (e.g., more than two) within the same block, where each partition is associated with a different block family. Each partition can include one or more written pages. Further, in alternative embodiments, block families can be associated with zones of data, which each zone can include one or more blocks of data.

Accordingly, the disclosed embodiments can be extended to or adapted for tracking block families written at differing granularities than just blocks.

(96) FIG. **10** illustrates an example machine of a computer system **1000** within which a set of instructions, for causing the machine to perform any one or more of the methodologies discussed herein, can be executed. In some embodiments, the computer system **1000** can correspond to a host system (e.g., the host system **120** of FIG. **1**) that includes, is coupled to, or utilizes a memory sub-system (e.g., the memory sub-system **110** of FIG. **1**) or can be used to perform the operations of a controller (e.g., to execute an operating system to perform operations corresponding to the block family manager **113** of FIG. **1**). In alternative embodiments, the machine can be connected (e.g., networked) to other machines in a LAN, an intranet, an extranet, and/or the Internet. The machine can operate in the capacity of a server or a client machine in client-server network environment, as a peer machine in a peer-to-peer (or distributed) network environment, or as a server or a client machine in a cloud computing infrastructure or environment.

(97) The machine can be a personal computer (PC), a tablet PC, a set-top box (STB), a Personal Digital Assistant (PDA), a cellular telephone, a web appliance, a server, a network router, a switch or bridge, or any machine capable of executing a set of instructions (sequential or otherwise) that specify actions to be taken by that machine. Further, while a single machine is illustrated, the term “machine” shall also be taken to include any collection of machines that individually or jointly execute a set (or multiple sets) of instructions to perform any one or more of the methodologies discussed herein.

(98) The example computer system **1000** includes a processing device **1002**, a main memory **1004** (e.g., read-only memory (ROM), flash memory, dynamic random access memory (DRAM) such as synchronous DRAM (SDRAM) or Rambus DRAM (RDRAM), etc.), a static memory **1010** (e.g., flash memory, static random access memory (SRAM), etc.), and a data storage system **1018**, which communicate with each other via a bus **1030**.

(99) Processing device **1002** represents one or more general-purpose processing devices such as a microprocessor, a central processing unit, or the like. More particularly, the processing device can be a complex instruction set computing (CISC) microprocessor, reduced instruction set computing (RISC) microprocessor, very long instruction word (VLIW) microprocessor, or a processor implementing other instruction sets, or processors implementing a combination of instruction sets. Processing device **1002** can also be one or more special-purpose processing devices such as an application specific integrated circuit (ASIC), a field programmable gate array (FPGA), a digital signal processor (DSP), network processor, or the like. The processing device **1002** is configured to execute instructions **1028** for performing the operations and steps discussed herein. The computer system **1000** can further include a network interface device **1012** to communicate over the network **1020**.

(100) The data storage system **1018** can include a machine-readable storage medium **1024** (also known as a computer-readable medium) on which is stored one or more sets of instructions **1028** or software embodying any one or more of the methodologies or functions described herein. The instructions **1028** can also reside, completely or at least partially, within the main memory **1004** and/or within the processing device **1002** during execution thereof by the computer system **1000**, the main memory **1004** and the processing device **1002** also constituting machine-readable storage media. The machine-readable storage medium **1024**, data storage system **1018**, and/or main memory **1004** can correspond to the memory sub-system **110** of FIG. **1**.

(101) In one embodiment, the instructions **1028** include instructions to implement functionality corresponding to the block family manager **113** of FIG. **1**. While the machine-readable storage medium **1024** is shown in an example embodiment to be a single medium, the term “machine-readable storage medium” should be taken to include a single medium or multiple media that store the one or more sets of instructions. The term “machine-readable storage medium” or “non-transitory computer-readable storage medium” shall also be taken to include any medium that is capable of storing or encoding a set of instructions for execution by the machine and that cause the machine to perform any one or more of the methodologies of the present disclosure. The term “machine-readable storage medium” or “non-transitory computer-readable storage medium” shall accordingly be taken to include, but not be limited to, solid-state memories, optical media, and magnetic media.

(102) Some portions of the preceding detailed descriptions have been presented in terms of algorithms and symbolic representations of operations on data bits within a computer memory. These algorithmic descriptions and representations are the ways used by those skilled in the data processing arts to most effectively convey the substance of their work to others skilled in the art. An algorithm is here, and generally, conceived to be a self-consistent sequence of operations leading to a desired result. The operations are those requiring physical manipulations of physical quantities. Usually, though not necessarily, these quantities take the form of electrical or magnetic signals capable of being stored, combined, compared, and otherwise manipulated. It has proven convenient at times, principally for reasons of common usage, to refer to these signals as bits, values, elements, symbols, characters, terms, numbers, or the like.

(103) It should be borne in mind, however, that all of these and similar terms are to be associated with the appropriate physical quantities and are merely convenient labels applied to these quantities. The present disclosure can refer to the action and processes of a computer system, or similar electronic computing device, that manipulates and transforms data represented as physical (electronic) quantities within the computer system's registers and memories into other data similarly represented as physical quantities within the computer system memories or registers or other such information storage systems.

(104) The present disclosure also relates to an apparatus for performing the operations herein. This apparatus can be specially constructed for the intended purposes, or it can include a general purpose computer selectively activated or reconfigured by a computer program stored in the computer. Such a computer program can be stored in a computer readable storage medium, such as, but not limited to, any type of disk including floppy disks, optical disks, CD-ROMs, and magnetic-optical disks, read-only memories (ROMs), random access memories (RAMs), EPROMs, EEPROMs, magnetic or optical cards, or any type of media suitable for storing electronic instructions, each coupled to a computer system bus.

(105) The algorithms and displays presented herein are not inherently related to any particular computer or other apparatus. Various general purpose systems can be used with programs in accordance with the teachings herein, or it can prove convenient to construct a more specialized apparatus to perform the method. The structure for a variety of these systems will appear as set forth in the description below. In addition, the present disclosure is not described with reference to any particular programming language. It will be appreciated that a variety of programming languages can be used to implement the teachings of the disclosure as described herein.

(106) The present disclosure can be provided as a computer program product, or software, that can include a machine-readable medium having stored thereon instructions, which can be used to program a computer system (or other electronic devices) to perform a process according to the present disclosure. A machine-readable medium includes any mechanism for storing information in a form readable by a machine (e.g., a computer). In some embodiments, a machine-readable (e.g., computer-readable) medium includes a machine (e.g., a computer) readable storage medium such as a read only memory (“ROM”), random access memory (“RAM”), magnetic disk storage media,

optical storage media, flash memory devices, etc.

(107) In the foregoing specification, embodiments of the disclosure have been described with reference to specific example embodiments thereof. It will be evident that various modifications can be made thereto without departing from the broader spirit and scope of embodiments of the disclosure as set forth in the following claims. The specification and drawings are, accordingly, to be regarded in an illustrative sense rather than a restrictive sense.

Claims

1. A system comprising: a memory device; and a processing device, operatively coupled to the memory device, the processing device to perform operations, comprising: initializing a block family associated with the memory device; initializing a timer associated with the block family; and in response to the timer reaching a soft closure value: performing a soft closure of the block family; performing a hard closure of the block family in response to a first of the timer reaching a hard closure value or the block family satisfying a hard closure criteria; detecting a first write amplification of a previously closed block family that employed the hard closure value; and increasing the hard closure value by an amount of time calculated to reduce a second write amplification, of the block family, to below a threshold write amplification.
2. The system of claim 1, wherein the operations further comprise setting the hard closure value of the timer based on at least one of a workload of the processing device, a reference temperature associated with the block family, or a performance expectation of the processing device.
3. The system of claim 1, wherein the operations further comprise setting the hard closure value of the timer as the soft closure value plus an additional threshold percentage of the soft closure value.
4. The system of claim 1, wherein the operations further comprise: after initializing the block family, responsive to beginning to program a block residing on the memory device, associating the block with the block family; and evaluating the hard closure criteria by: determining an amount of time until filling the block completely with data being received from a host system; and deciding whether to finish programming the block based on the amount of time and a remainder of time left until the timer reaches the hard closure value.
5. The system of claim 4, wherein the operations further comprise, in response to deciding to not finish programming the block, one of: leaving empty an unwritten portion of the block; or writing dummy data to the unwritten portion of the block.
6. The system of claim 4, wherein the operations further comprise, in response to deciding to not finish programming the block: creating a first partition of a written portion of the block; associating the first partition with the block family; and creating a second partition of an unwritten portion of the block for association with a new block family.
7. The system of claim 4, wherein, in response to deciding to finish programming the block, the operations further comprise finish programming the block with data received from the host system before performing the hard closure of the block family.
8. The system of claim 1, wherein the operations further comprise: detecting a bit error rate of a previously closed block family that employed the hard closure value, wherein the bit error rate is above a threshold acceptable bit error rate; and reducing the hard closure value by an amount of time calculated to reduce the bit error rate to below the threshold acceptable bit error rate.
9. A method comprising: initializing, by a processing device, a block family associated with a memory device; initializing a timer associated with the block family; and in response to the timer reaching a soft closure value: performing a soft closure of the block family; performing a hard closure of the block family in response to a first of the timer reaching a hard closure value or the block family satisfying a hard closure criteria; detecting a first write amplification of a previously closed block family that employed the hard closure value; and increasing, by the processing device, the hard closure value by an amount of time calculated to reduce a second write amplification, of

the block family, to below a threshold write amplification.

10. The method of claim 9, further comprising setting the hard closure value of the timer based on at least one of a workload of the processing device, a reference temperature associated with the block family, or a performance expectation of the processing device.

11. The method of claim 9, further comprising setting the hard closure value of the timer as the soft closure value plus an additional threshold percentage of the soft closure value.

12. The method of claim 9, further comprising: after initializing the block family, responsive to beginning to program a block residing on the memory device, associating the block with the block family; and evaluating the hard closure criteria by: determining an amount of time until filling the block completely with data being received from a host system; and deciding whether to finish programming the block based on the amount of time and a remainder of time left until the timer reaches the hard closure value.

13. The method of claim 12, further comprising, in response to deciding to not finish programming the block, one of: leaving empty an unwritten portion of the block; or writing dummy data to the unwritten portion of the block.

14. The method of claim 12, further comprising, in response to deciding to not finish programming the block: creating a first partition of a written portion of the block; associating the first partition with the block family; and creating a second partition of an unwritten portion of the block for association with a new block family.

15. The method of claim 12, wherein, in response to deciding to finish programming the block, the method further comprising to finish programming the block with the data received from the host system before performing the hard closure of the block family.

16. The method of claim 9, further comprising: detecting a bit error rate of a previously closed block family that employed the hard closure value, wherein the bit error rate is above a threshold acceptable bit error rate; and reducing the hard closure value by an amount of time calculated to reduce the bit error rate to below the threshold acceptable bit error rate.

17. A non-transitory computer-readable storage medium that stores instructions, which when executed by a processing device of a memory device, cause the processing device to perform operations comprising: initializing a block family associated with a memory device; initializing a timer associated with the block family; and in response to the timer reaching a soft closure value: performing a soft closure of the block family; performing a hard closure of the block family in response to a first of the timer reaching a hard closure value or the block family satisfying a hard closure criteria; detecting a first write amplification of a previously closed block family that employed the hard closure value; and increasing the hard closure value by an amount of time calculated to reduce a second write amplification, of the block family, to below a threshold write amplification.

18. The non-transitory computer-readable storage medium of claim 17, wherein the operations further comprise setting the hard closure value of the timer based on at least one of a workload of the processing device, a reference temperature associated with the block family, or a performance expectation of the processing device.

19. The non-transitory computer-readable storage medium of claim 17, wherein the operations further comprise setting the hard closure value of the timer as the soft closure value plus an additional threshold percentage of the soft closure value.

20. The non-transitory computer-readable storage medium of claim 17, wherein the operations further comprise: after initializing the block family, responsive to beginning to program a block residing on the memory device, associating the block with the block family; and evaluating the hard closure criteria by: determining an amount of time until filling the block completely with data being received from a host system; and deciding whether to finish programming the block based on the amount of time and a remainder of time left until the timer reaches the hard closure value.
